

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи № 6
з курсу “Життєвий цикл програмного забезпечення. Управління ІТ
проектами”
на тему: “Версіонування програмного забезпечення.
Децентралізована система контролю версій”

Виконав:

Студент. гр. ФсП-41 Фіняк Олег

Перевірив:

Франів В.А.

Мета роботи: Полягає в глибшому розумінні концепції децентралізованих систем контролю версій та їх ролью в процесі розробки програмного забезпечення. Ознайомитись з основними принципами та перевагами використання децентралізованих систем контролю версій порівняно з централізованими аналогами. Створити власний децентралізований репозиторій використовуючи сервіс GitHub

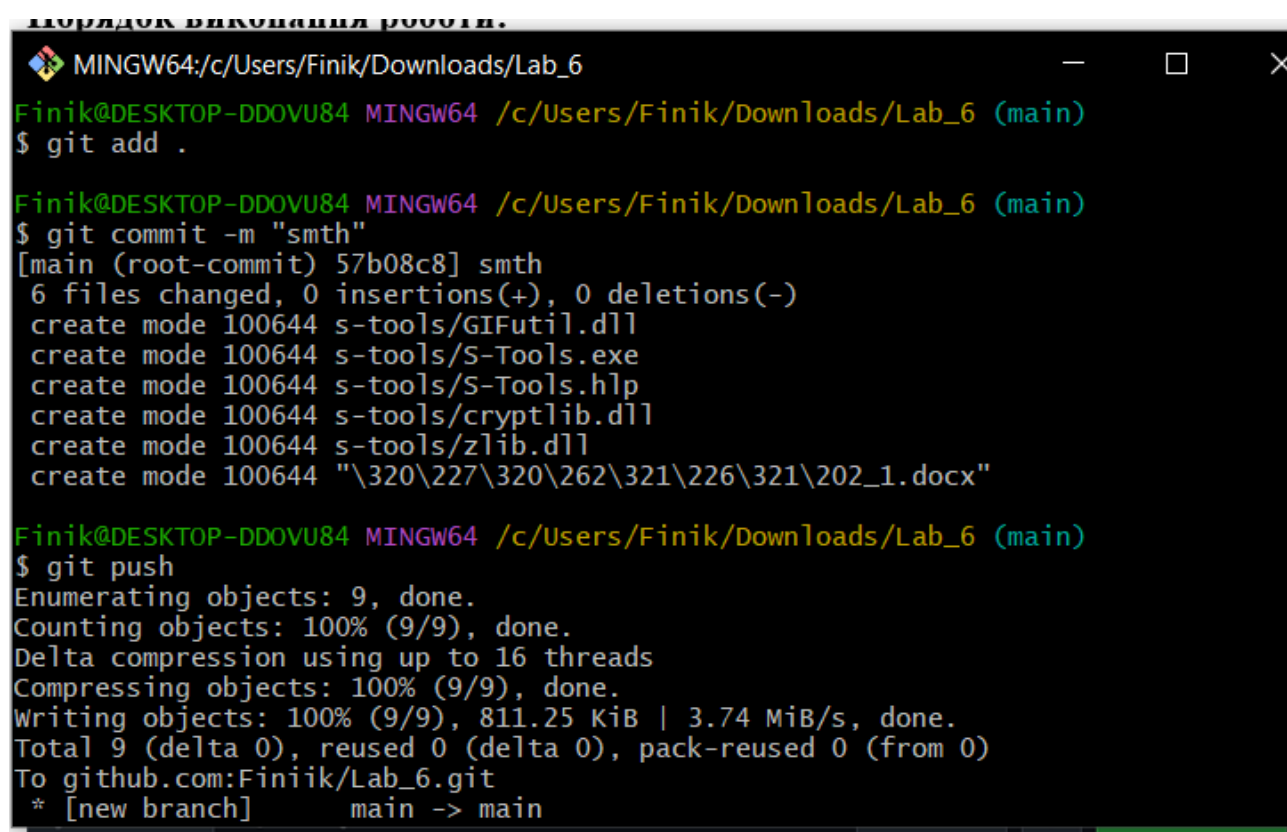
Вимоги до результатів виконання лабораторної роботи:

1. Створити локальний репозиторій з проектом з попередніх лабораторних робіт, створити віддалений репозиторій використовуючи сервіс GitHub.
2. Описати ключові стадії версіонування проекту з використання децентралізованого репозиторію.
3. Проаналізувати плюси та мінуси використання децентралізованих систем контролю версій.

Порядок виконання роботи:

1. Освоїти вище наведений теоретичний матеріал.
2. Створити профіль у GitHub.
3. Створіть публічний репозиторій.
4. Додайте проект з попередньої лабораторної роботи до віддаленого репозиторію.

Порядок виконання роботи.



```
MINGW64:/c/Users/Finik/Downloads/Lab_6
Finik@DESKTOP-DDOVU84 MINGW64 /c/Users/Finik/Downloads/Lab_6 (main)
$ git add .

Finik@DESKTOP-DDOVU84 MINGW64 /c/Users/Finik/Downloads/Lab_6 (main)
$ git commit -m "smth"
[main (root-commit) 57b08c8] smth
6 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 s-tools/GIFutil.dll
create mode 100644 s-tools/S-Tools.exe
create mode 100644 s-tools/S-Tools.hlp
create mode 100644 s-tools/cryptlib.dll
create mode 100644 s-tools/zlib.dll
create mode 100644 "\\320\\227\\320\\262\\321\\226\\321\\202_1.docx"

Finik@DESKTOP-DDOVU84 MINGW64 /c/Users/Finik/Downloads/Lab_6 (main)
$ git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 16 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 811.25 KiB | 3.74 MiB/s, done.
Total 9 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:Finik/Lab_6.git
 * [new branch]      main -> main
```

5. Створіть гілку "brach-1" на локальному репозиторії і зробіть декілька комітів.

6. Запускайте гілку "brach-1" до ремоутного репозиторію.

```
Finik@DESKTOP-DDOVU84 MINGW64 /c/Users/Finik/Downloads/Lab_6 (main)
$ git checkout -b brach-1
Switched to a new branch 'brach-1'

Finik@DESKTOP-DDOVU84 MINGW64 /c/Users/Finik/Downloads/Lab_6 (brach-1)
$ git status
On branch brach-1
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    LOLlo1.docx

nothing added to commit but untracked files present (use "git add" to track)

Finik@DESKTOP-DDOVU84 MINGW64 /c/Users/Finik/Downloads/Lab_6 (brach-1)
$ git add .

Finik@DESKTOP-DDOVU84 MINGW64 /c/Users/Finik/Downloads/Lab_6 (brach-1)
$ git commit -m "Test commit for branch-1"
[brach-1 0d6a5e6] Test commit for branch-1
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 LOLlo1.docx

Finik@DESKTOP-DDOVU84 MINGW64 /c/Users/Finik/Downloads/Lab_6 (brach-1)
$ git push origin brach-1
```

7. Створіть пул ріквест.

 brach-1 had recent pushes 4 seconds ago [Compare & pull request](#)

 main ▾

[Go to file](#) [+](#) [<> Code ▾](#)

 **Finiik** smth 57b08c8 · 1 minute ago 

 s-tools

smth

1 minute ago

 3bit_1.docx

smth

1 minute ago

 [README](#)

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base: **main** ← compare: **branch-1**

✓ **Able to merge.** These branches can be automatically merged.

Add a title *

Test commit for branch-1

Add a description

Write

Preview

H B I ≡ <> 🔗 | ☰ ☷ ☹ | 📎 @ ↗ ↶ ✎

Its all ok

📖 Markdown is supported

📎 Paste, drop, or click to add files

Create pull request

8. Зробіть ревью пул ріквесту та замерджіть його.

Test commit for branch-1 #1 ✎

🔗 Open Finiik wants to merge 1 commit into **main** from **branch-1** 📄

💬 Conversation 0 🔗 Commits 1 📋 Checks 0 📁 Files changed 1

 **Finiik** commented now

Owner ⋮

Its all ok



  Test commit for branch-1

0d6a5e6



No conflicts with base branch

Merging can be performed automatically.

Merge pull request

You can also merge this with the command line. [View command line instructions](#).

Commit message

Merge pull request #1 from Finiik/brach-1

Extended description

Test commit for branch-1

This commit will be authored by 115735330+Finiik@users.noreply.github.com.

Confirm merge

Cancel

Still in progress? [Convert to draft](#)

Pull request successfully merged and closed

You're all set — the `branch-1` branch can be safely deleted.

Delete branch

main

Go to file

+

Code

 **Finiik**

Merge pull request #1 from Finiik/brach-1

d4d4e46 · now

s-tools	smth	2 minutes ago
LOllol.docx	Test commit for branch-1	1 minute ago
Звіт_1.docx	smth	2 minutes ago

Висновок:

На основі виконаної роботи можна виділити наступні переваги децентралізованих систем контролю версій (ДСКВ) у порівнянні з централізованими (ЦСКВ):

Відсутність єдиної точки відмови (SPOF): У ДСКВ (Git) кожен розробник має повну копію сховища з усією історією на своєму комп'ютері. Якщо сервер GitHub тимчасово недоступний, робота не зупиняється, а у разі збою на сервері будь-який локальний клон може слугувати повноцінною резервною

копією. У централізованих системах (наприклад, SVN) вихід сервера з ладу паралізує роботу всієї команди.

Підтримка нелінійного розвитку: Децентралізована архітектура та механізм Pull Requests роблять процес галуження (branching) дуже легким та безпечним, що дозволяє паралельно вести тисячі гілок і ефективно керувати кодом навіть без прямого доступу до запису в головний репозиторій (через систему "форків").

Література:

6. Scott Chacon, Ben Straub. Pro Git. Apress. 2014. 456 p.
7. Jon Loeliger, Matthew McCullough. Version Control with Git. O'Reilly Media. 2012. 456 p.
8. Ryan Hodson. Git Essentials. Packt Publishing. 2015. 194 p.
9. Roger Dudler. Git Pocket Guide. O'Reilly Media. 2013. 234 p.
10. Seth Robertson, Steve Oualline. Practical Git: For Everyday Use. No Starch Press. 2014. 300 p